

Unit 3 Lab Date Instructions – Questions To Answer and Things to Fix

3-1: Text Input Write a program that prompts the user to enter the date in **mm/dd/yyyy format**. The program should use a **single input statement** to accept the date, storing each piece of the date into an appropriate variable. Demonstrate that the input was obtained correctly by outputting the month, day, and year to the screen. Sample output: Enter (date (mm/dd/yyyy): 02/08/2011 Month entered: 2 Day entered: 8 Year entered: 2011 Challenge: Although the user entered 02, C++ drops the 0 as insignificant when the program outputs the value. Revise your program to always output two-digit months and days. For example, 02 should output as 02 and 10 should output as 10 (not 010).

I used the above instructions to ask AI to write some really nice and simple code to do this for me and I am going to provide you with AI's solution.

BUT – it is your job to explain the following to me about AI's date solution that I am attaching:

- (1) Why is namespace not needed in AI's solution ?
 - a. Standard library functionality gets written into header files like `iostream`. Inside those files, is a directly nested namespace declaration for 'std' or "standard" namespace. When we provide a pre-processor statement like `#include <iostream>` we are telling the compiler to go grab all the code within the `iostream` header file and paste it in-place of the pre-processor statement. However the functions, classes, and variables are all encapsulated within the `std` namespace scope. Thus we must explicitly state that we would either like local access to the namespace like `'using namespace std;'` or we must be explicit with any individual function, class, or variable that we would like to use. In the case of `'cin'` and `'cout'` we can do this using the scope resolution operator `'::'`. This essentially tells the compiler, look for the thing to my right inside the thing on my left. So we use `std::cin` to say "within 'std' locate and use the 'cin' function.
- (2) If I enter the value "2Z15L2026" I get the output of 02 day 05 month and 2026 as year so the program does not care I disobeyed and did not use a slash – why does this work ?
 - a. The program as it had been written accepted any input character in-place of the `slash1`, and `slash2` variables when performing the assignment statement. Thus since `'Z'` and `'L'` are by definition character datatype objects, they were taken.

Unit 3 Lab Date Instructions – Questions To Answer and Things to Fix

(3) How could you change the code so a slash was always the character required without breaking something else – give an example.

- a. The way to resolve this issue is to look inside the user input and validate that two instances of '/' can be located at the correct location in the character of the given input sequence.

```
// Check 1) if and 2) how many '/' characters exist in line
// Then 3) if the '/' are located in the correct places within given format (mm/dd/yy) (3 & 6)
// Then break user input via 'line' into substrings l1 = mm | l2 = dd | l3 == yyyy
bool checkSlash(std::string& line, std::string& l1, std::string& l2, std::string& l3){

    int count = 0;
    int stringLocation = 0;

    // Count occurrences of '/' in line after loop(s) run
    // Don't run loop if no '/' are found
    while(line.find('/', stringLocation)!=line.npos){
        stringLocation = line.find('/', stringLocation) + 1;
        count++;}

    // If 2 '/' are found in character locations of 3 & 6 of 'line'
    // Break input into associated substrings for mm, dd, and yyyy
    // Then return true
    if(count == 2 && line[2] == '/' && line[5] == '/'){
        l1 = line.substr(0, 2);
        l2 = line.substr(3, 2);
        l3 = line.substr(6);
        return true;}

    // Otherwise return false
    else{return false;}

}
b. }
```

(4) If enter the value "abcdefg" the error routine works but if I enter the value "123456" the program just hangs – **why is that** and how can you fix it – explain and show me with corrected code.

- a. Lets talk about `cin >> var`, it's a binary operator meaning it takes 2 parameters [e.g. `(p1)operator>>(p2)`]. When we call this operator and it sees an istream object it immediately attempts to find an overload resolution for the given datatype of p2 in association with the p1 object. When doing this the operator>> can be ran in two distinct ways. As a member function of istream (`cin`) or as a non-member function. The distinction is meaningful since the non-member function handles characters, and string arrays it simply will attempt to assign the address value at the istream object buffer sequence and assign it to the address of the p2 variable. That's after it runs sentry of course. However, when p2 is an int, float-type, or bool it runs the

Unit 3 Lab Date Instructions – Questions To Answer and Things to Fix

formatted/member function version. In this version, the operator<> runs `get_num`, and subsequently `std::get_num()::do_get()` after `do()` calls it. When `do_get()` runs it does 3 things. First it performs conversion specification set by i/o flags and the declaration type of `p2`. Second, the character extraction happens and if the first buffer sequence character that is currently selected is a char type, it fails and it sets an iostate flag of `std::io_base::failbit`, it then puts a non-zero value in the buffer, sets the destination variable (`p2 (int)`) equal to 0, this doesn't even raise any exceptions. It just silently triggers and then every subsequent extraction operation fails as soon as sentry sees `failbit`. Additionally, if the input is in fact an integer `do_get()` will take the entire sequence of integers starting at the first one, until it finds the end of stream or a non-eligible character `val`. That's why the assignment statement hangs when we put one large integer value into it. However if the integer is too big it will make `do_get()` set `failbit` again since it won't fit into the max size for the int variable. In either case, after <> fails or succeeds it returns the stream object.

- b. The next 2 pages shows the portion of my code that represents my input validation handling for these stated issues:

Unit 3 Lab Date Instructions – Questions To Answer and Things to Fix

```
(5) // Find out if each character in range of string 's' is between 0-9
(6) // Return 0 if not; or greater than 0 value if is
(7) bool isCharInt(std::string& s){
(8)
(9)     int i = 0;
(10)         int len = s.length();
(11)
(12)         // While iterator 'i' is less than len(length of string 's')
(13)         while (i<len){
(14)
(15)             // If character at location 'i' @ 's' is not a digit
(16)             // Return false | trigger invalidation
(17)             if(std::isdigit((unsigned char)s[i])==0){
(18)                 return false;}
(19)             i++;} // Iterate
(20)         // Or return true if valid
(21)         return true;
(22)     }
(23)
(24)     // Check if a given string 'str' is a digit via isCharInt()
(25)     // Check if the integer value of 'str' is greater than 0
(26)     int checkInt(std::string& str){
(27)
(28)         // Is it a digit & Is it greater than 0?
(29)         if(isCharInt(str) && std::stoi(str, 0)>0){
(30)             // YES = return value of integer value of 'str'
(31)             return std::stoi(str, 0);}
(32)         // NO = return 0
(33)         else{return 0;}
(34)     }
(35)
(36)     // Check if l1(mm) = maximum of 12 | l2(dd) = max of 31
(37)     // Check that l1, l2, l3 are digits and are greater than 1 via checkInt() returns not 0
(38)     bool checkCalVal(std::string& l1, std::string& l2, std::string& l3){
(39)
(40)         int monInYr = 12;
(41)         int dayInMo = 31;
(42)
(43)         // if checkInt returns validation condition (return value > 0)
(44)         // if l1 <= 12
(45)         // if l2 <= 31
```

Unit 3 Lab Date Instructions – Questions To Answer and Things to Fix

```
(46)         if(checkInt(l1)!=0 && checkInt(l2) != 0 && checkInt(l3) != 0 &&
    checkInt(l1)<=monInYr && checkInt(l2)<=dayInMo){
(47)             //return true if validated
(48)             return true;}
(49)         // otherwise return false
(50)         else{return false;}
(51)     }
(52)
(53)     // Validation orchestrator
(54)     // Takes a memory address poointer to 'line'
(55)     bool validate(std::string& line){
(56)
(57)         // Initializes l1(mm), l2(dd), l3(yyyy)
(58)         std::string l1, l2, l3;
(59)
(60)         // validate length of 10 | Has no whitespaces | Has a count of 2 '/'(s); substring
    line into l1, l2, l3 ->
(61)         // ->| line (as l1,l2,l3) Contains only integers > 0; l1(mm)<=12; l2(dd)<=31
(62)         if(checkLen(line) && !checkSpaces(line) && checkSlash(line, l1, l2, l3) &&
    checkCalVal(l1, l2, l3)){
(63)             // If meets validation
(64)             return true;}
(65)         // IF invalid
(66)         else{return false;}
(67)     }
(68)
```